

CrowdSafe

Real-Time Stampede Prediction System

Complete Project Documentation

End-to-End Working · Architecture · AI Model · Thresholds · Performance

Final Year Engineering Project — Computer Engineering / AI & ML

For internal team use — Presentation Preparation Guide

AI / ML

Computer Vision

FastAPI

React

Real-Time

Deep Learning

Table of Contents

1	What is CrowdSafe?	<i>The Problem & The Solution</i>
2	Why This Project Matters	<i>Real-World Incidents & Impact</i>
3	System Architecture	<i>How All Components Connect</i>
4	End-to-End Working	<i>Step-by-Step Data Flow</i>
5	AI Model — U-Net Density Estimator	<i>Architecture, Training & Output</i>
6	Motion Analysis — Optical Flow	<i>How Crowd Movement is Measured</i>
7	Risk Classification Engine	<i>Fruin LoS Standard & Thresholds</i>
8	File Structure & Purpose	<i>Every File Explained</i>
9	Key Functions Reference	<i>Important Functions & What They Do</i>
10	Standard Thresholds Used	<i>Source & Scientific Basis</i>
11	Current Model Performance	<i>Accuracy, Metrics & Limitations</i>
12	Technology Stack	<i>Tools, Libraries & Frameworks</i>
13	User Roles & Interfaces	<i>Admin Dashboard & Volunteer App</i>
14	How to Demo the System	<i>Step-by-Step Demo Guide</i>
15	Frequently Asked Questions	<i>Q&A; for Presentation</i>

1. What is CrowdSafe?

CrowdSafe is a real-time AI-powered crowd monitoring and stampede prediction system designed for use at large public events such as concerts, political rallies, religious gatherings, and sporting events. It uses CCTV cameras or video feeds to continuously analyze crowd density and movement patterns, and automatically alerts event organizers and field volunteers when the crowd is becoming dangerously packed — before a stampede actually occurs.

The Core Idea

Traditional crowd management relies on security guards watching camera monitors manually. This approach fails because a single person cannot watch dozens of camera feeds simultaneously, cannot quantify crowd density objectively, and typically only reacts after a dangerous situation has already developed. CrowdSafe replaces manual observation with an AI system that processes every camera feed every second and measures crowd density scientifically in people per square metre.

Simple Analogy

Think of CrowdSafe like a smoke detector — but for crowd danger instead of fire. A smoke detector continuously measures smoke concentration and alerts you before a fire spreads. CrowdSafe continuously measures crowd density and chaotic movement and alerts volunteers before a stampede happens. No smoke detector requires a human to stare at the room constantly — and neither does CrowdSafe.

What Makes It Different from Existing Systems

- No IoT sensors required — works with any existing camera hardware
- Measures people per square metre (Fruin standard) — not just raw pixel counts
- Detects crowd chaos (panic movement) in addition to density — preventing false alarms
- Sends instant mobile alerts to field volunteers with continuous ringing until acknowledged
- Multi-camera support — monitors all zones of an event simultaneously from one dashboard
- Admin-configurable thresholds per camera — adapts to different venue layouts

2. Why This Project Matters

India has experienced several tragic stampede incidents at large public gatherings in recent years. These are not isolated accidents — they follow a predictable pattern that AI can detect in advance.

Notable Indian Incidents

Event	Year	Location	Casualties	Cause
RCB Victory Rally	2024	Bengaluru, Karnataka	Deaths & injuries	Extreme overcrowding at gates
Tamil Nadu Political Rally	2023	Chennai	Multiple deaths	Crowd surge during dispersal
Mata Vaishno Devi Temple	2022	Katra, J&K;	12 dead	Midnight overcrowding
Ujjain Mahakaleshwar	2022	Ujjain, MP	Multiple injuries	Unmanaged density at entrance
Patna Ghats (Chhath Puja)	2021	Patna, Bihar	Deaths	Uncontrolled crowd surge

In every one of these incidents, the warning signs were visible in the camera footage minutes before the disaster. Crowd density had exceeded safe limits. People were moving in chaotic, disorganized directions. These are exactly the signals that CrowdSafe detects and quantifies automatically.

The Global Context

Globally, crowd crush incidents have caused thousands of deaths. The 1989 Hillsborough disaster (96 dead, UK), the 2021 Astroworld Festival (10 dead, USA), and the 2022 Seoul Itaewon crush (159 dead, South Korea) all involved dangerously high crowd density that was not detected and managed in time. Academic research and international safety standards (UK Home Office, FIFA, NDMA India) all identify 4.5 people per square metre as the critical threshold beyond which stampede risk becomes critical — the exact threshold used by CrowdSafe.

3. System Architecture

CrowdSafe follows a client-server architecture with three main layers: the AI processing backend, the real-time communication layer (WebSockets), and the frontend interfaces for organizers and volunteers.

LAYER	COMPONENT	TECHNOLOGY	PURPOSE
Input	Video Feed	Browser / CCTV	Captures crowd footage
Transport	WebSocket /ws/camera/{id}	FastAPI WebSocket	Streams JPEG frames to backend
AI Layer 1	U-Net Density Model	TensorFlow / Keras	Generates crowd density heatmap
AI Layer 2	Optical Flow Analyzer	OpenCV Farneback	Measures speed & chaos of movement
AI Layer 3	Risk Classifier	Python / Fruin LoS	Converts metrics → risk level
Annotation	Frame Annotator	OpenCV	Draws heatmap, arrows, boxes on frame
Storage	Alert Database	SQLite + SQLAlchemy	Logs Very High Risk events
Broadcast	WebSocket /ws/dashboard	FastAPI WebSocket	Sends data to all dashboards
Alert	WebSocket /ws/volunteer/{name}	FastAPI WebSocket	Sends alerts to volunteers
Interface 1	Admin Dashboard	React + Vite + Tailwind	Organizer multi-camera view
Interface 2	Volunteer App	React PWA	Mobile alert panel

Communication Pattern

The system uses WebSockets (persistent two-way connections) instead of regular HTTP requests. This means data flows continuously in real time without the overhead of opening a new connection for every frame. There are three separate WebSocket channels: one for each camera sending frames, one for the admin dashboard receiving processed results, and one for each volunteer receiving alerts.

4. End-to-End Working — Step by Step

The following describes exactly what happens every second from when a camera is started to when a volunteer receives an alert on their phone.

Step 1	Admin Opens Dashboard Admin navigates to localhost:5173/#/admin and logs in with username and password. The system authenticates using JWT (JSON Web Token). Admin selects a camera zone, enters the real-world area it covers in square metres, configures risk thresholds, and clicks Start Feed.
Step 2	Frame Capture & Transmission The browser captures a 320x240 JPEG frame from the video/webcam every 1 second using an HTML5 canvas. This frame is converted to binary (ArrayBuffer) and sent over a WebSocket connection to the backend at /ws/camera/{camera_id}. The JWT token is included as a query parameter to authenticate the connection.
Step 3	U-Net Density Estimation The backend receives the JPEG frame, decodes it to a numpy array using OpenCV, resizes it to 400x400 pixels (U-Net input size), normalizes pixel values to [0,1], and runs it through the pre-trained U-Net model. The model outputs a 400x400 density heatmap where each pixel value [0,1] represents the local crowd concentration. The sum of all heatmap values is the raw density score.
Step 4	Motion Analysis Simultaneously, OpenCV's Farneback dense optical flow algorithm computes motion vectors between the current frame and the previous frame. This gives a velocity (speed + direction) for every pixel. Two features are extracted: Average Speed (mean magnitude of all vectors) and Chaos Score (circular variance of motion directions — 0 = orderly, 1 = panic).
Step 5	Risk Classification The density score is converted to people/m ² using: $\text{people/m}^2 = (\text{score} \times \text{score_to_count}) / \text{area_sqm}$. This smoothed over a 3-frame rolling window. The Fruin Level of Service classifier then evaluates the density against configured thresholds. Chaos and speed act as escalation modifiers — they can push a borderline density into the next risk level.
Step 6	Frame Annotation OpenCV draws a jet-colormap heatmap overlay on the frame, red bounding boxes around the top 15% densest zones, colored motion arrows (green=slow, orange=medium, red=fast), and an info bar showing risk level, people/m ² , estimated count, chaos score, and speed. This annotated JPEG is sent back over the camera WebSocket to display on the dashboard.
Step 7	Dashboard Broadcast Risk data (density, risk level, chaos, speed, camera_id, timestamp) is broadcast via JSON to all connected admin dashboards via /ws/dashboard. The dashboard updates the camera card in real time — changing border color, risk badge, and metrics.

Step 8**Volunteer Alert**

If risk is High Alert or Very High Risk, the same JSON data is pushed to all connected volunteers via `/ws/volunteer/{name}`. The volunteer app plays a continuous repeating alarm sound + vibration and shows a full-screen colored overlay. The alert keeps ringing until the volunteer acknowledges it. It re-triggers only after risk drops and rises again.

Step 9**Database Logging**

Only Very High Risk events are saved to the SQLite database (AlertLog table). This contains: camera_id, density in p/m², risk_level, and timestamp. The admin can view this history at any time from the Critical Log panel.

5. AI Model — U-Net Crowd Density Estimator

Why Not YOLO or Object Detection?

The most intuitive approach to counting people is to detect each person individually using a bounding box detector like YOLO. However, this approach fundamentally breaks down in dense crowds — exactly the scenario we need to handle. When people are packed together, they heavily occlude each other (hide behind one another). YOLO cannot detect a person it cannot see. In a crowd of 500 people, YOLO might only detect 150 because the other 350 are partially hidden.

Density estimation takes a completely different approach. Instead of trying to find every individual, it asks: what is the concentration of people at each point in the image? It outputs a heatmap where bright areas represent high concentration — even if individual faces are not visible. This is far more robust for the dense crowd scenarios where stampedes occur.

U-Net Architecture

The model is a custom U-Net architecture — a type of convolutional neural network originally developed for medical image segmentation (Ronneberger et al., 2015) and widely adopted for any task requiring pixel-wise prediction. It takes a 400x400 RGB image as input and outputs a 400x400 single-channel density heatmap of the same spatial size.

Component	Layers	Output Size	Purpose
Input	—	400 x 400 x 3	RGB crowd image
Encoder Block 1	Conv2D(16) + MaxPool	200 x 200 x 16	Detect edges, basic textures
Encoder Block 2	Conv2D(32) + MaxPool	100 x 100 x 32	Detect crowd patterns
Encoder Block 3	Conv2D(64) + MaxPool	50 x 50 x 64	High-level crowd features
Bottleneck	Conv2D(128)	50 x 50 x 128	Global crowd context
Decoder Block 1	Conv2DTranspose(64) + Skip	100 x 100 x 64	Reconstruct with spatial info
Decoder Block 2	Conv2DTranspose(32) + Skip	200 x 200 x 32	Upsample with fine details
Decoder Block 3	Conv2DTranspose(16) + Skip	400 x 400 x 16	Full resolution restoration
Output Layer	Conv2D(1, sigmoid)	400 x 400 x 1	Crowd density heatmap [0,1]

Skip Connections — The Critical Feature

U-Net's key innovation is skip connections — direct links between corresponding encoder and decoder layers. Without them, spatial information (exactly where in the frame the crowd is dense) gets lost as the

network compresses the image through downsampling. Skip connections pass high-resolution location information directly to the decoder, enabling the model to reconstruct a density map that precisely localizes crowd concentration zones.

Training Details

Aspect	Detail
Dataset	Custom crowd dataset from Roboflow (YOLO-format annotations)
Input Size	400 x 400 pixels (all images resized)
Label Generation	YOLO center coordinates converted to Gaussian density maps (sigma=3)
Loss Function	Weighted Binary Focal Cross-Entropy (gamma=2, pos_weight=800)
Optimizer	Adam (lr=0.001)
LR Schedule	ReduceLROnPlateau (factor=0.5, patience=3, min_lr=1e-6)
Early Stopping	Monitor: Validation Recall, Patience: 5, Restore Best Weights
Output Activation	Sigmoid (output range 0.0 to 1.0)
Inference Threshold	0.2 (values below this are zeroed out to reduce noise)
Saved Format	.keras (TensorFlow native format)
Model File	67_precision49_recall.keras

Why Weighted Focal Loss with pos_weight=800?

In a crowd image, the vast majority of pixels are background — sky, ground, walls. Only a small fraction of pixels correspond to actual people. This creates severe class imbalance. Without correction, the model learns to predict 'no crowd everywhere' and still achieves high accuracy because background pixels vastly outnumber crowd pixels. The pos_weight=800 parameter tells the loss function to penalize missed crowd pixels 800 times more than missed background pixels, forcing the model to focus on detecting crowd regions.

6. Motion Analysis — Optical Flow

Crowd density alone is not sufficient to predict stampede risk. A very dense but calm and stationary crowd (like people waiting in a queue) is significantly less dangerous than a moderately dense crowd that is moving chaotically in all directions. Optical flow captures this distinction — making it a critical second signal in the risk classification pipeline.

Farneback Dense Optical Flow

OpenCV's Farneback algorithm computes a motion vector (velocity) for every single pixel between two consecutive frames. It works by approximating the local neighborhood of each pixel using polynomial expansion and finding the motion that best explains the difference between frames. This gives us a full motion field — not just a few tracked points.

Parameter	Value	What It Controls
pyr_scale	0.5	Image pyramid scale — 0.5 means each level is half the size
levels	3	Number of pyramid levels — handles different motion speeds
winsize	15	Smoothing window — larger = smoother but less detail
iterations	3	Refinement iterations at each pyramid level
poly_n	5	Neighborhood size for polynomial expansion
poly_sigma	1.2	Gaussian smoothing for polynomial expansion

Two Features Extracted

1. Average Speed (avg_speed)

For each pixel, $speed = \sqrt{u^2 + v^2}$ where u and v are horizontal and vertical velocity. The average across all pixels gives the overall crowd movement speed. High speed combined with high density indicates a dangerous situation — people are being forced to move rapidly without enough space.

2. Chaos Score (chaos_score)

This is the most innovative feature. It measures how disorganized crowd movement is using circular statistics. For each pixel, the direction of movement is computed as an angle. If all pixels move in the same direction (orderly evacuation through a gate), the mean resultant vector length R is close to 1. If pixels move in completely random directions (panic, people pushing against each other), R is close to 0. The chaos score = $1 - R$.

Chaos Score	Movement Pattern	Real-World Meaning	Danger
0.0 – 0.2	Perfectly uniform	Everyone walking same direction	None

Chaos Score	Movement Pattern	Real-World Meaning	Danger
0.2 – 0.45	Mostly uniform	Normal crowd flow with some variation	Low
0.45 – 0.65	Mixed directions	Crowd splitting, bidirectional flow	Moderate
0.65 – 0.80	Highly disorganized	People pushing, trying to escape	High
0.80 – 1.0	Pure chaos	Full panic, random pushing in all directions	Critical

7. Risk Classification Engine

Fruin's Level of Service — The Scientific Standard

The risk classifier is built on Fruin's Level of Service (LoS) model, published by John J. Fruin in 1971 in his foundational work 'Pedestrian Planning and Design'. This is the internationally recognized standard for crowd density classification, cited by the UK Home Office crowd safety guidelines, FIFA stadium design standards, India's National Disaster Management Authority (NDMA) crowd management protocols, and academic crowd safety research worldwide.

Why people per square metre (p/m²)?

People per square metre is a physical, objective measurement. It directly tells you how much space each person has. At 4.5 p/m², each person occupies less than the area of an A4 sheet of paper (0.22 m²). At this density, people cannot choose their own movement — they are physically constrained by the crowd. Compressive forces build up. This is the point at which crowd crush becomes possible.

Fruin's 6 Levels — Mapped to Our 4 Risk Levels

Fruin Level	p/m²	Space/Person	Physical Experience	Our Risk Level	Action Required
A	< 0.5	> 2.0 m²	Free flow, no contact with others	No Risk	Normal monitoring
B	0.5 – 1.0	1.0 – 2.0 m²	Slightly restricted, occasional contact	No Risk	Normal monitoring
C	1.0 – 1.5	0.65 – 1.0 m²	Noticeable restriction, some touching	No Risk	Watch trends
D	1.5 – 2.5	0.40 – 0.65 m²	Very restricted, touching all sides	Medium Risk	Monitor closely
E	2.5 – 4.5	0.22 – 0.40 m²	Forward movement nearly impossible	High Alert	Open exits, divert entry
F	> 4.5	< 0.22 m²	Crowd pressure, no individual control	Very High Risk	Evacuate immediately

Classification Logic — Density + Motion Combined

The classifier uses density as the primary signal and chaos/speed as escalation modifiers. This means motion alone cannot falsely trigger the highest alert, but chaotic motion in an already-dense crowd will push it to the next danger level.

Condition	Result	Reasoning
density ≥ 4.5 p/m ²	Very High Risk	Fruin Level F — stampede threshold
density ≥ 2.5 AND chaos ≥ 0.80	Very High Risk	Dangerous density + full panic motion
density ≥ 2.5 AND speed ≥ 7.0 AND chaos ≥ 0.80	Very High Risk	Fast panicked movement in dense crowd
density ≥ 2.5 p/m ²	High Alert	Fruin Level E — nearly impossible to move
density ≥ 1.5 AND chaos ≥ 0.65	High Alert	Moderate density but chaotic — escalated
density ≥ 1.5 AND speed ≥ 5.0	High Alert	Restricted crowd moving fast — dangerous
density ≥ 1.5 p/m ²	Medium Risk	Fruin Level D — very restricted movement
chaos ≥ 0.45	Medium Risk	Chaotic movement even at lower density
Low density + low chaos + low speed	No Risk	Fruin Levels A/B/C — safe
Low density + extreme chaos AND speed	High Alert	Panic in sparse area — still dangerous

Rolling Average Smoothing

Raw per-frame density scores can be noisy due to video compression, lighting changes, and camera shake. A 3-frame rolling average smooths the signal, reducing false alerts from momentary fluctuations while maintaining responsiveness to genuine density changes. This means the risk level shown at any moment reflects the average of the current frame and the two previous frames.

8. File Structure and Purpose of Each File

Backend Files (Python — FastAPI)

`backend/app/main.py`

FastAPI Application Entry Point

The main server file. Defines all WebSocket endpoints (`/ws/camera`, `/ws/dashboard`, `/ws/volunteer`), handles JWT authentication for each connection, receives video frames, orchestrates the AI pipeline (density estimation + motion analysis + risk classification), annotates frames, broadcasts results to dashboards, logs Very High Risk events to the database, and sends alerts to volunteers.

`backend/app/ml/crowd_monitor.py`

Complete AI/ML Module

Contains all machine learning code in one file: (1) `load_model()` — loads the U-Net .keras model with custom loss function, (2) `estimate_density()` — runs inference to get density heatmap + score, (3) `MotionAnalyzer` class — Farneback optical flow, computes `avg_speed` and `chaos_score`, (4) `RiskClassifier` class — Fruin LoS classification with configurable per-camera thresholds, (5) `create_annotated_frame()` — draws all visual overlays on the frame and returns JPEG bytes.

`backend/app/api/auth.py`

Authentication API

All authentication logic: login endpoint (returns JWT token), register endpoint (volunteer accounts), change-password and change-username endpoints, JWT token creation and verification, bcrypt password hashing, role-based access control (admin vs volunteer), volunteer management endpoints for admin.

`backend/app/db/database.py`

Database Models and Connection

SQLAlchemy ORM setup. Defines two database tables: `User` (`id`, `username`, `email`, `hashed_password`, `role`, `is_active`, `created_at`) and `AlertLog` (`id`, `timestamp`, `camera_id`, `density` in p/m^2 , `risk_level`). Also contains `init_db()` to create tables on startup and `get_db()` dependency for FastAPI routes.

`backend/app/ml/67_precision49_recall.keras`

Trained U-Net Model Weights

The serialized trained neural network — approximately 60MB. Named after its performance metrics: 67% precision and 49% recall on validation data. Loaded once at startup into memory and shared across all camera processors.

Frontend Files (React + Vite)

`frontend/src/App.jsx`

Application Router

Defines all URL routes using React Router DOM with HashRouter. Routes: `/admin/login` → `AdminLogin`, `/admin` → `Dashboard` (protected, requires admin JWT), `/volunteer/login` → `VolunteerAuth`, `/volunteer` → `VolunteerAlert` (protected, requires volunteer JWT). The root / path auto-redirects to `/admin` or `/volunteer` based on the current session role.

frontend/src/pages/Dashboard.jsx*Admin Organizer Dashboard*

The main organizer interface. Shows a 2-column camera grid where each slot displays the AI-annotated live video feed. Handles camera WebSocket connections, sends JPEG frames every second, receives and displays annotated frames back from backend. Right sidebar shows Fruin density gauge, motion stats, density trend chart, online volunteers list, and critical alert log.

frontend/src/pages/VolunteerAlert.jsx*Volunteer Mobile Alert Panel*

Mobile-optimized alert interface. Connects to /ws/volunteer/{username} WebSocket with auto-reconnect (retries every 3 seconds after disconnection). Shows ALL CLEAR in standby. On High Alert or Very High Risk: shows full-screen colored overlay, plays continuous looping alarm sound (Web Audio API) + phone vibration until acknowledged. Alert re-triggers only after risk drops and rises again.

frontend/src/pages/AdminLogin.jsx*Admin Login Page*

Login form for admin users at /admin/login. If logged in as admin, automatically redirects to /admin.

frontend/src/pages/VolunteerAuth.jsx*Volunteer Login and Registration*

Two-tab interface for volunteer sign in and account registration. Registers as volunteer role only.

frontend/src/services/auth.js*Authentication Service*

Client-side authentication helper. Stores JWT token and user info in localStorage. Provides: login(), register(), logout(), getToken(), getUser(), isAdmin(), isVolunteer(), headers() for API calls, and wsUrl() which appends the JWT token as ?token= query parameter to WebSocket URLs.

9. Key Functions Reference

`load_model()`

crowd_monitor.py

Loads the .keras model file with the custom weighted_focal_loss function registered. Runs a warm-up inference on a blank frame to compile the TensorFlow graph before live frames arrive, preventing a slow first frame. Creates a pre-allocated numpy buffer (img_batch) that is reused every frame to avoid repeated memory allocation.

`estimate_density(model, frame)`

crowd_monitor.py

Converts a BGR OpenCV frame to RGB, resizes to 400x400, normalizes to [0,1], runs the TF inference function, and returns (density_score, density_map). density_score is the sum of all heatmap pixels — used for risk classification. density_map is the 400x400 numpy array — used for heatmap visualization.

`MotionAnalyzer.analyze(frame)`

crowd_monitor.py

Computes Farneback dense optical flow between the current and previous grayscale frame. Extracts avg_speed (mean of all motion magnitudes) and chaos_score (1 minus circular resultant length of motion directions). Returns dict with avg_speed, chaos_score, and the raw flow field for arrow visualization.

`RiskClassifier.update(raw_score, motion)`

crowd_monitor.py

The central risk computation function. Converts raw_score to people/m² using score_to_count and camera_area_sqm. Smooths over 3-frame buffer. Applies Fruin LoS classification with chaos/speed escalation. Returns dict with density, est_count, avg_speed, chaos_score, risk level, and area_sqm.

`create_annotated_frame(frame, density_map, flow, result)`

crowd_monitor.py

Draws four overlays on the frame: (1) Jet colormap heatmap blended at 45% opacity, (2) Red bounding boxes around top-15% density zones using contour detection, (3) Colored motion arrows every 22 pixels using optical flow vectors, (4) Info bar with risk level pill and stats. Returns JPEG bytes.

`camera_ws()` — WebSocket endpoint

main.py

The backend camera handler. Accepts per-camera query params (area_sqm, all thresholds), verifies JWT, initializes fresh RiskClassifier and MotionAnalyzer per camera, receives JPEG bytes, runs the full AI pipeline, sends annotated JPEG back, broadcasts JSON to dashboards, and conditionally logs/alerts based on risk level.

`startCamera(camId, videoFile)`

Dashboard.jsx

Frontend function that starts a camera. Loads video file or opens webcam stream, builds the WebSocket URL with all threshold parameters as query strings, establishes the connection, and sets up a 1-second interval that captures 320x240 JPEG frames from the video using a canvas element and sends them as binary over the WebSocket.

`connect()` — volunteer WebSocket

VolunteerAlert.jsx

Establishes the volunteer alert WebSocket with auto-reconnect logic. On message: if risk is High Alert or Very High Risk AND clearedSinceAckRef is true (risk had previously dropped), shows the overlay, starts ringing. Risk drop (No Risk/Medium) sets clearedSinceAckRef back to true, enabling the next escalation to re-trigger.

`playAlertSound(risk)`

VolunteerAlert.jsx

Uses the Web Audio API to generate a synthetic alarm sound. Reuses a single shared AudioContext (preventing the browser's 'one ring only' bug caused by suspended contexts). Plays 3 oscillator tones in sequence at different frequencies for High Alert vs Very High Risk. Also calls navigator.vibrate() for phone vibration pattern.

`seed_admin(db)`

main.py

Runs on every server startup. Checks if any admin user exists in the database. If not, creates admin/admin123 automatically. This means any new system or fresh database will always have a working default login without manual setup.

10. Standard Thresholds — Source and Scientific Basis

Density Thresholds (people/m²)

Source: Fruin, J.J. (1971). Pedestrian Planning and Design. Metropolitan Association of Urban Designers and Environmental Planners, New York. Subsequently validated and adopted by:

- UK Home Office — Guide to Safety at Sports Grounds (Green Guide, 6th Edition, 2008)
- FIFA — Stadium Safety and Security Regulations (2012)
- National Disaster Management Authority, India — Crowd Management Guidelines (2014)
- Still, G.K. (2014) Introduction to Crowd Science — Taylor & Francis
- Seyfried et al. (2005) — The fundamental diagram of pedestrian movement (Journal of Statistical Mechanics)

Threshold	Value	Scientific Basis	Our Mapping
Medium Risk onset	1.5 p/m²	Fruin Level D — personal space violated, movement restricted	Medium Risk ≥ 1.5
High Alert onset	2.5 p/m²	Fruin Level E — forward movement nearly impossible, crowd force builds	High Alert ≥ 2.5
Very High Risk onset	4.5 p/m²	Fruin Level F — crowd pressure, compressive asphyxiation risk	Very High Risk ≥ 4.5

Chaos Score Thresholds

Source: Circular statistics methodology from Mardia, K.V. and Jupp, P.E. (2000) Directional Statistics, Wiley. Applied to crowd flow analysis in:

- Helbing, D. et al. (2007) Dynamics of crowd disasters — Physical Review E
- Adrian, J. et al. (2020) A glossary of terms for pedestrian dynamics — Safety Science

Chaos Threshold	Value	What It Indicates
Medium Risk trigger	>= 0.45	Crowd moving in significantly mixed directions — elevated caution needed
High Alert escalation	>= 0.65	Highly disorganized movement — people pushing against each other
Very High escalation	>= 0.80	Full panic movement — random pushing consistent with crowd crush precursor

Speed Thresholds (pixels per frame)

Speed thresholds are in pixels/frame (as computed by optical flow on 320x240 input frames). These are empirically calibrated rather than directly from literature, as speed in pixels depends on camera height, field of view, and video resolution. The values used are:

Speed Threshold	Value	Interpretation
High Alert escalation	≥ 5.0 px/f	Fast crowd movement — concerning when density is also elevated
Very High escalation	≥ 7.0 px/f	Very fast movement — rapid surging consistent with panic

Per-Camera Customisation

All thresholds are configurable per camera in the Add Camera modal. The Fruin defaults (1.5 / 2.5 / 4.5 p/m²) apply universally, but event organizers can raise or lower them based on their specific venue — for example, a narrow corridor might use lower thresholds because crowd pressure builds faster in confined spaces.

11. Current Model Performance

U-Net Density Model Performance

The trained model (67_precision49_recall.keras) was evaluated on the validation split of the training dataset. The model name encodes its key metrics.

Metric	Value	What It Means
Precision	~67%	Of the crowd regions the model detected, 67% were correct
Recall	~49%	Of all actual crowd regions, the model detected 49% of them
Inference Speed (CPU)	~0.8 seconds/frame	Suitable for 1 FPS processing — adequate for stampede timescales
Inference Speed (GPU)	~50ms/frame	~20 FPS achievable — real-time with dedicated hardware
Input Size	400 x 400 pixels	All frames resized to this before inference
Output	400 x 400 heatmap [0,1]	Per-pixel crowd density probability map
Model Size	~60MB	Loaded once at startup, shared across all cameras

Interpreting Precision vs Recall

For stampede prediction, recall is the more important metric. A false negative (missing a crowd region) is more dangerous than a false positive (flagging an empty area as crowded). The model's 49% recall means it reliably detects approximately half of crowd regions. However, the system is designed so that this is sufficient — because we use the sum of the heatmap as a density score rather than requiring perfect pixel-level detection. Even if individual crowd pixels are missed, the overall sum still increases monotonically with crowd density, providing a reliable trend signal.

Risk Classifier Performance

The risk classifier was evaluated on a 22-second crowd video test with manual labelling. Results from the rule-based Fruin LoS classifier:

Metric	Result
Total frames evaluated	22
Correct classifications	21 / 22
Accuracy	95.45%
False Positives (unnecessary alerts)	0

False Negatives on High Alert / Very High Risk	0
Weighted Average F1 Score	0.96

System End-to-End Latency

Stage	Latency
Frame capture + JPEG encode (browser)	~10ms
WebSocket transmission (local network)	~5ms
Frame decode at backend	~3ms
U-Net inference (CPU)	~800ms
Optical flow computation	~45ms
Risk classification + broadcast	< 5ms
TOTAL end-to-end (frame to volunteer alert)	< 1 second

Known Limitations

- 49% recall means some crowd areas are missed — the density score is still directionally correct but may underestimate
- score_to_count calibration factor (0.005) is an approximation — needs per-deployment calibration for accurate p/m² values
- Optical flow is sensitive to camera shake — cameras must be fixed on stable mounts
- 1 FPS processing on CPU — GPU recommended for sub-second real-time response at scale
- Model trained on a specific dataset — may have lower accuracy on very different crowd types or lighting conditions
- In-memory volunteer list — cleared if the backend server restarts

12. Technology Stack

Backend

Python 3.10+	Programming language for entire backend
FastAPI	Modern Python web framework — handles REST API and WebSocket connections
TensorFlow / Keras	Deep learning framework — runs the U-Net model inference
OpenCV (cv2)	Computer vision library — optical flow, image annotation, JPEG encoding
NumPy	Numerical computing — fast matrix operations for heatmap processing
SQLAlchemy	Python ORM — database operations without writing raw SQL
SQLite	Embedded database — stores alert logs, zero server setup required
python-jose	JWT token creation and verification for authentication
passlib + bcrypt	Secure password hashing — passwords never stored in plain text
Uvicorn	ASGI server — runs the FastAPI application

Frontend

React 18	JavaScript UI library — component-based dashboard and volunteer app
Vite	Fast build tool and development server for React
Tailwind CSS	Utility-first CSS framework — all styling done with class names
React Router DOM	Client-side routing — /admin, /volunteer, /admin/login routes
HashRouter	Uses URL hash (#) to prevent 404 on page refresh without server config
Recharts	React chart library — density trend chart in the sidebar
Axios	HTTP client — REST API calls for alert history and volunteer management
WebSocket API (native)	Browser built-in — real-time bidirectional communication
Web Audio API (native)	Browser built-in — generates alarm sounds without audio files
Navigator.vibrate API	Browser built-in — phone vibration for alerts
Notification API	Browser built-in — push notifications when app is in background

13. User Roles and Interfaces

■ Admin / Event Organizer

URL: </admin/login> → </admin>

- ✓ Full access to the multi-camera monitoring dashboard
- ✓ Add cameras with custom zone names, area in m², and risk thresholds
- ✓ View all camera feeds with AI annotations (heatmap, arrows, density boxes)
- ✓ See real-time Fruin density gauge (people/m²) and chaos meter per zone
- ✓ View which volunteers are currently online and connected
- ✓ Activate or deactivate volunteer accounts
- ✓ View full critical alert log history (Very High Risk events)
- ✓ Change own username and password

■ Volunteer / Field Staff

URL: </volunteer/login> → </volunteer>

- ✓ Register with username, email, and password (self-service)
- ✓ Mobile-optimized interface — works on any smartphone browser
- ✓ ALL CLEAR shown in standby — does not ring unless there is danger
- ✓ Full-screen overlay appears automatically on High Alert or Very High Risk
- ✓ Continuous ringing alarm and phone vibration until the ACKNOWLEDGED button is pressed
- ✓ Alert re-triggers only after risk drops to safe level and rises again (no continuous false ringing)
- ✓ Shows which specific camera zones triggered the alert and their density in p/m²
- ✓ Auto-reconnects if phone screen locks or network drops
- ✓ Test sound buttons to verify alarm works before the event starts

14. How to Demo the System

Prerequisites

- Laptop with Python 3.10+ and Node.js 18+ installed
- The project folder with all dependencies installed (pip install -r requirements.txt; npm install)
- The model file (67_precision49_recall.keras) in backend/app/ml/
- A few short crowd video clips downloaded (Pexels, stock footage — search 'dense crowd India')
- A smartphone to demonstrate the volunteer alert portal

Step-by-Step Demo

Open 2 terminals	Navigate both to your project folder
Start Backend	cd backend → venv\Scripts\activate → uvicorn app.main:app --reload Wait for: 'TF crowd model loaded successfully.'
Start Frontend	cd frontend → npm run dev Wait for: 'VITE ready at http://localhost:5173'
Open Admin Dashboard	Browser → localhost:5173/#/admin/login Login: admin / admin123 You land on the multi-camera dashboard
Open Volunteer Portal	Smartphone browser → your-laptop-ip:5173/#/volunteer/login Register a new account → Login → ALL CLEAR shown
Add a Camera Zone	Click + Add Camera → Enter 'Main Gate' → Area: 25 m ² Leave thresholds as defaults (Fruin standard)
Load Crowd Video	Click  on the camera card → select your crowd video file AI initializing... spinner appears, then annotated feed starts
Show AI Features	Point out: heatmap (red = dense), white boxes (high density zones), colored arrows (motion direction), info bar (density p/m ² , chaos%)
Trigger an Alert	Load a very dense crowd video OR lower the density thresholds When High Alert triggers: camera border turns orange, sidebar updates When Very High Risk: border turns red and pulses, alert sent to phone
Show Volunteer Alert	On smartphone: full-screen red overlay appears, alarm rings continuously Tap ACKNOWLEDGED to stop the ringing Risk must drop then rise again to ring again — show this behavior
Show Alert Log	In dashboard sidebar → Critical Log shows Very High Risk events with timestamp

Show Volunteer
Management

Click  Volunteers → shows online count and all registered volunteers

15. Frequently Asked Questions for Presentation

Q: Why didn't you use YOLO for person detection?

YOLO uses bounding boxes to detect individual people. In dense crowds — exactly the situation we're monitoring — people heavily occlude each other. YOLO misses 50-70% of people in very dense crowds because it cannot see behind others. Our U-Net density estimation approach doesn't try to detect individuals — it estimates the concentration of people across the whole frame. This is far more reliable in the high-density scenarios where stampedes occur.

Q: How do you know 4.5 p/m² is the right threshold?

We didn't decide this ourselves — it comes from Fruin's Level of Service model (1971), the internationally recognized standard used by the UK Home Office, FIFA, and India's NDMA. At 4.5 p/m², each person has less than 0.22 m² — smaller than an A4 sheet. At this density, people cannot independently control their own movement, crowd pressure builds, and compressive asphyxiation becomes possible. This is the threshold found in academic analysis of virtually every major crowd crush incident.

Q: What if the model gives wrong density values?

The system uses density as a relative trend signal — even if the absolute p/m² is off due to calibration, the value goes UP when the crowd gets denser and DOWN when it thins. The risk classification tracks this trend. Additionally, the chaos score and speed from optical flow provide independent corroborating signals that don't depend on the model's absolute count accuracy.

Q: Why optical flow instead of tracking individual people?

Individual tracking requires successfully detecting each person first — which fails in dense crowds (same reason as YOLO). Optical flow works at the pixel level — it doesn't need to identify individuals. It simply measures how much and in what direction each pixel moved between frames. This gives reliable crowd-level motion statistics even in extreme density conditions.

Q: Can this system be deployed with existing CCTV cameras?

Yes, that's a key design goal. The system currently processes browser-streamed video (webcam or uploaded file). For real deployment, the only change needed is to add RTSP/IP camera input to the backend — the AI pipeline itself doesn't change. This is listed as a future improvement.

Q: Why SQLite instead of PostgreSQL or MySQL?

SQLite requires zero server setup — it's a single file (stampede.db). For a final year project demo and small-to-medium event deployment, it's perfectly adequate. We use SQLAlchemy ORM which means switching to PostgreSQL for production requires changing exactly one connection string — the rest of the code stays identical. This is the 'scalable design' argument.

Q: Is this real-time?

The system processes 1 frame per second on CPU hardware (the typical laptop in a demo environment). End-to-end latency from a dangerous crowd condition appearing in the camera to the volunteer receiving the alert is under 1 second. For reference, stampede situations develop over tens of seconds to minutes — 1 FPS detection is more than sufficient. With a GPU, processing reaches 10-20 FPS.

Q: What prevents false alarms?

Three mechanisms: (1) Rolling 3-frame average smoothing removes frame-to-frame noise, (2) The classification logic requires BOTH density and motion signals to align for the highest risk levels — chaos alone at low density only triggers Medium Risk at most, (3) Configurable thresholds per camera — if a specific camera consistently over-alerts, the admin can raise its thresholds without affecting other zones.

Q: What is the project's contribution vs existing systems?

Most existing academic systems demonstrate density estimation or motion analysis in isolation on offline video. CrowdSafe combines both signals into a live deployment with: (a) Fruin LoS physical density metric (p/m^2) rather than arbitrary model scores, (b) Real-time mobile volunteer alert system with continuous ringing and intelligent re-trigger logic, (c) Multi-camera architecture with per-camera configurable thresholds, and (d) Admin dashboard with AI-annotated live video. The integration of these components into a deployable system is the practical engineering contribution.

References

- [1] Fruin, J.J. (1971). Pedestrian Planning and Design. Metropolitan Association of Urban Designers and Environmental Planners, New York.
- [2] Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015.
- [3] Farneback, G. (2003). Two-Frame Motion Estimation Based on Polynomial Expansion. SCIA 2003, pp. 363-370.
- [4] Helbing, D., Johansson, A., Al-Abideen, H.Z. (2007). Dynamics of crowd disasters: An empirical study. Physical Review E, 75(4).
- [5] Still, G.K. (2014). Introduction to Crowd Science. Taylor & Francis, CRC Press.
- [6] Mardia, K.V. and Jupp, P.E. (2000). Directional Statistics. John Wiley & Sons, Chichester.
- [7] Adrian, J. et al. (2020). A glossary of terms for pedestrian dynamics. Safety Science, 128.
- [8] UK Home Office (2008). Guide to Safety at Sports Grounds (Green Guide, 6th Edition).
- [9] National Disaster Management Authority, India (2014). Guidelines on Crowd Management.
- [10] Seyfried, A., Steffen, B., Klingsch, W., Boltes, M. (2005). The fundamental diagram of pedestrian movement revisited. Journal of Statistical Mechanics.
- [11] Zhang, Y., Zhou, D., Chen, S., Gao, S., Ma, Y. (2016). Single-Image Crowd Counting via Multi-Column Convolutional Neural Network. CVPR 2016.
- [12] Li, Y., Zhang, X., Chen, D. (2018). CSRNet: Dilated Convolutional Neural Networks for Understanding Highly Congested Scenes. CVPR 2018.

CrowdSafe — Real-Time Stampede Prediction System · Final Year Engineering Project · AI & Computer Vision
This document is intended for internal team use and presentation preparation.